

Assignment 2

Data Quality Assessment & Preprocessing

ARTI308 - Machine Learning- Fedaa Alsabea- 2230003675

Task 1 Identify data quality issues in the dataset:

The image displays two screenshots of a Jupyter Notebook interface, showing the process of identifying and converting data types in a dataset.

Top Screenshot: The notebook is titled "Untitled-ARTI406 - Assignment 2.ipynb". The cell content shows the task instructions and the first step of data quality assessment.

```
Task 1 Identify data quality issues in the dataset.
```

```
2. Data Quality Assessment
```

```
2.1 Check Data Types
```

The output of the `df.dtypes` command is shown:

Column	dtype
Inspection ID	int64
DBA Name	object
AKA Name	object
License #	float64
Facility Type	object
Risk	object
Address	object
City	object
State	object
Zip	float64
Inspection Date	object
Inspection Type	object
Results	object
Violations	object
Latitude	float64
Longitude	float64
Location	object
Historical Wards 2003-2015	float64
Zip Codes	float64
Community Areas	float64
Census Tracts	float64
Wards	float64
dtype:	object

Bottom Screenshot: The notebook shows the next step in the data quality assessment process.

```
2.2 Convert Incorrect Data Types
```

```
df['Inspection Date'] = pd.to_datetime(df['Inspection Date'])
```

The output of the `df.dtypes` command after conversion is shown:

Column	dtype
Inspection ID	int64
DBA Name	object
AKA Name	object
License #	float64
Facility Type	object
Risk	object
Address	object
City	object
State	object
Zip	float64
Inspection Date	datetime64[ns]
Inspection Type	object
Results	object
Violations	object
Latitude	float64
Longitude	float64
Location	object
Historical Wards 2003-2015	float64
Zip Codes	float64
Community Areas	float64
Census Tracts	float64
Wards	float64
dtype:	object

File Edit Selection View Go Run Terminal Help

Untitled-ART1406 - Assignment 2.ipynb

C > Users > fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance. > M4 Data Reduction - P

Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline

Python 3.10.11

Longitude float64
Location object
Historical Wards 2003-2015 float64
Zip Codes float64
Community Areas float64
Census Tracts float64
Wards float64
dtype: object

2.3 Check Missing Values

```
df.isna().sum()
```

[5] ✓ 0.0s Python

Inspection ID	0
DBA Name	0
AKA Name	2458
License #	17
Facility Type	4768
Risk	69
Address	0
City	139
State	42
Zip	50
Inspection Date	0
Inspection Type	1
Results	0
Violations	52266
Latitude	690
Longitude	690
Location	690
Historical Wards 2003-2015	196825
Zip Codes	196825
Community Areas	196825
Census Tracts	196825
Wards	196825
dtype:	int64

File Edit Selection View Go Run Terminal Help

Untitled-ART1406 - Assignment 2.ipynb

C > Users > fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance. > M4 Data Reduction - P

Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline

Python 3.10.11

Address 0
City 139
State 42
Zip 50
Inspection Date 0
Inspection Type 1
Results 0
Violations 52266
Latitude 690
Longitude 690
Location 690
Historical Wards 2003-2015 196825
Zip Codes 196825
Community Areas 196825
Census Tracts 196825
Wards 196825
dtype: int64

Several columns contain missing values.

The columns with the highest number of missing values are:

- Violations
- Historical Wards 2003-2015
- Zip Codes
- Community Areas
- Census Tracts
- Wards

2.4 Check Duplicate Records

```
df.duplicated().sum()
```

[5] ✓ 0.3s Python

np.int64(149)

The dataset was checked for data quality issues by examining data types, missing values, and duplicate records.

First, the data types were checked. The Inspection Date column was originally stored as an object, so it was converted to datetime format.

Then, missing values were identified. Several columns contained missing values, especially Violations, Historical Wards 2003-2015, Zip Codes, Community Areas, Census Tracts, and Wards.

Finally, duplicate records were checked. The dataset contained 149 duplicate records.

Task 2 Apply one missing value strategy and explain why:

The first screenshot shows the notebook interface with the title "Task 2 Apply one missing value strategy and explain why." and the subtitle "Handling Missing Values". The section "3.1 Detect Missing Values" is active. The code cell contains the following Python code:

```
df_missing = df.copy()
df_missing.isnull().sum()
```

The output of the code is a series of counts for each column, indicating the number of missing values. The output is as follows:

```
Inspection ID      0
DBA Name           0
AKA Name          2458
License #          17
Facility Type      4768
Risk              69
Address           69
City             139
State            42
Zip              50
Inspection Date    0
Inspection Type    1
Results           0
Violations        52266
Latitude          690
Longitude         690
Location          690
Historical Wards 2003-2015 196825
Zip Codes         196825
Community Areas   196825
Census Tracts     196825
Wards             196825
dtype: int64
```

The second screenshot shows the notebook interface with the same title and subtitle. The code cell contains the following Python code:

```
print("Dataset Shape:", df.shape)
print("Dataset Copy Shape:", df_missing.shape)
```

The output of the code is:

```
Dataset Shape: (196825, 22)
Dataset Copy Shape: (196825, 22)
```

The next code cell contains the following Python code:

```
df_missing.head(10)
```

The output of the code is a table showing the first 10 rows of the dataset with missing values. The table has 16 columns: Inspection ID, DBA Name, AKA Name, License #, Facility Type, Risk, Address, City, State, Zip, Inspection Date, Inspection Type, Results, Violations, Latitude, and Longitude. The table shows the following data:

	Inspection ID	DBA Name	AKA Name	License #	Facility Type	Risk	Address	City	State	Zip	Inspection Date	Inspection Type	Results	Violations	Latitude	Longitude
0	2352683	CHIPOTLE MEXICAN GRILL	CHIPOTLE MEXICAN GRILL	2670642.0	Restaurant	Risk 1 (High)	1025 W ADDISON ST	CHICAGO	IL	60613.0	2019-12-04	License Re-Inspection	Pass w/ Conditions	3. MANAGEMENT, FOOD EMPLOYEE AND CONDITIONAL E...	41.947118	-87.65538
1	2352734	CHILI'S T-I	CHILI'S (T1-B14)	34169.0	Restaurant	Risk 1 (High)	11601 W TOUHY AVE	CHICAGO	IL	60666.0	2019-12-04	Canvass	Pass	10. ADEQUATE HANDWASHING SINKS PROPERLY SUPPLI...	42.008536	-87.91442
2	2352713	NICK'S FIRE GRILL STEAK & LEMONADE INC.	NICK'S FIRE GRILL STEAK & LEMONADE INC.	2699090.0	NaN	Risk 1 (High)	2900 W 63RD ST	CHICAGO	IL	60629.0	2019-12-04	License	Not Ready	NaN	41.779190	-87.69589
3	2352701	TAQUERIA BLUE LINE INC.	NaN	2703436.0	Restaurant	Risk 1 (High)	3401 W BELMONT AVE	CHICAGO	IL	60618.0	2019-12-04	License	Pass w/ Conditions	3. MANAGEMENT, FOOD EMPLOYEE AND CONDITIONAL E...	41.939168	-87.71254

FileEditSelectionViewGoRunTerminalHelp<-->Search

Untitled-ART1406 - Assignment 2.ipynb

C:\Users> fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance: > M4 Data Reduction - P

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.10.11

3.2 Apply Missing Value Strategy 2: Median Imputation

[24]df_missing.head(10)Python

	Inspection ID	DBA Name	AKA Name	License #	Facility Type	Risk	Address	City	State	Zip	Inspection Date	Inspection Type	Results	Violations	Latitude	Longitude
0	2352683	CHIPOTLE MEXICAN GRILL	CHIPOTLE MEXICAN GRILL	2670642.0	Restaurant	Risk 1 (High)	1025 W ADDISON ST	CHICAGO	IL	60613.0	2019-12-04	License Re-Inspection	Pass w/ Conditions	3. MANAGEMENT, FOOD EMPLOYEE AND CONDITIONAL E...	41.947118	-87.65538
1	2352734	CHILI'S T-1	CHILI'S (T1-814)	34169.0	Restaurant	Risk 1 (High)	11601 W TOUHY AVE	CHICAGO	IL	60666.0	2019-12-04	Canvass	Pass	10. ADEQUATE HANDWASHING SINKS PROPERLY SUPPLI...	42.008536	-87.91442
2	2352713	NICK'S FIRE GRILL STEAK & LEMONADE INC.	NICK'S FIRE GRILL STEAK & LEMONADE INC.	2699090.0	NaN	Risk 1 (High)	2900 W 63RD ST	CHICAGO	IL	60629.0	2019-12-04	License	Not Ready	NaN	41.779190	-87.69589
3	2352701	TAQUERIA BLUE LINE INC.	NaN	2703436.0	Restaurant	Risk 1 (High)	3401 W BELMONT AVE	CHICAGO	IL	60618.0	2019-12-04	License	Pass w/ Conditions	3. MANAGEMENT, FOOD EMPLOYEE AND CONDITIONAL E...	41.939168	-87.71254
4	2352727	PORTAGE PARK DAY NURSERY	MOSAIC EARLY CHILDHOOD ACADEMY	2215815.0	Children's Services Facility	Risk 1 (High)	5332-5334 W ADDISON ST	CHICAGO	IL	60641.0	2019-12-04	Canvass	Pass	NaN	41.946065	-87.76072
		AMARIT	AMARIT			Risk 1	600 S				2019-12-	Canvass				

Spaces: 4 CRLF Cell 55 of 55

FileEditSelectionViewGoRunTerminalHelp<-->Search

Untitled-ART1406 - Assignment 2.ipynb

C:\Users> fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance: > M4 Data Reduction - P

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.10.11

```
df_imputed = df.copy()

cols_to_drop = [
    'Historical Wards 2003-2015',
    'Zip Codes',
    'Community Areas',
    'Census Tracts',
    'Wards'
]

df_imputed = df_imputed.drop(columns=cols_to_drop)

df_imputed['License #'] = df_imputed['License #'].fillna(df_imputed['License #'].median())
df_imputed['Zip'] = df_imputed['Zip'].fillna(df_imputed['Zip'].median())
df_imputed['Latitude'] = df_imputed['Latitude'].fillna(df_imputed['Latitude'].median())
df_imputed['Longitude'] = df_imputed['Longitude'].fillna(df_imputed['Longitude'].median())

df_imputed['AKA Name'] = df_imputed['AKA Name'].fillna(df_imputed['AKA Name'].mode()[0])
df_imputed['Facility Type'] = df_imputed['Facility Type'].fillna(df_imputed['Facility Type'].mode()[0])
df_imputed['Risk'] = df_imputed['Risk'].fillna(df_imputed['Risk'].mode()[0])
df_imputed['City'] = df_imputed['City'].fillna(df_imputed['City'].mode()[0])
df_imputed['State'] = df_imputed['State'].fillna(df_imputed['State'].mode()[0])
df_imputed['Inspection Type'] = df_imputed['Inspection Type'].fillna(df_imputed['Inspection Type'].mode()[0])
df_imputed['Violations'] = df_imputed['Violations'].fillna(
    df_imputed['Violations'].mode()[0]
)

df_imputed['Location'] = df_imputed['Location'].fillna(
    df_imputed['Location'].mode()[0]
)

df_imputed.isnull().sum()
```

(9) 0.5s Python

Inspection ID 0
DBA Name 0

Spaces: 4 CRLF Cell 55 of 55

```
df_imputed.isnull().sum()
```

```
df_imputed.head(10)
```

	Inspection ID	DBA Name	AKA Name	License #	Facility Type	Risk	Address	City	State	Zip	Inspection Date	Inspection Type	Results	Violations	Latitude	Longitude
0	2352683	CHIPOTLE MEXICAN GRILL	CHIPOTLE MEXICAN GRILL	2670642.0	Restaurant	Risk 1 (High)	1025 W ADDISON ST	CHICAGO	IL	60613.0	2019-12-04	License Re-Inspection	Pass w/ Conditions	3. MANAGEMENT, FOOD EMPLOYEE AND CONDITIONAL E...	41.947118	-87.65538
1	2352734	CHILI'S T-I	CHILI'S (T1-B14)	34169.0	Restaurant	Risk 1 (High)	11601 W TOUHY AVE	CHICAGO	IL	60666.0	2019-12-04	Canvass	Pass	10. ADEQUATE HANDWASHING SINKS PRONPERIV	42.008536	-87.91442

The dataset contained missing values in several columns. A missing value analysis was performed to identify the affected features.

To handle missing values, a median imputation strategy was applied to numerical columns such as License #, Zip, Latitude, and Longitude. The median was selected because it is less sensitive to extreme values and provides a robust estimate of the center of the data.

For categorical columns such as AKA Name, Facility Type, Risk, City, State, Inspection Type, and Location, the mode (most frequent value) was used to replace missing entries.

Some columns contained a very large number of missing values, including Historical Wards 2003-2015, Zip Codes, Community Areas, Census Tracts, and Wards. These columns were removed because most of their values were missing and they would contribute little useful information to the analysis.

After applying the imputation strategy and removing highly incomplete columns, no missing values remained in the dataset.

Reason for Choosing Median Strategy:

Median imputation was selected because numerical variables may contain skewed distributions and outliers. Unlike the mean, the median is not heavily affected by extreme values, making it a more reliable replacement for missing numerical data.

Task 3 Detect and handle outliers using IQR:

Task 3 Detect and handle outliers using IQR.

Handling Outliers

```
df_imputed.describe()
```

	Inspection ID	License #	Zip	Inspection Date	Latitude	Longitude
count	1.968250e+05	1.968250e+05	196825.000000	196825	196825.000000	196825.000000
mean	1.449379e+06	1.601167e+06	60628.740998	2014-12-21 22:28:44.305347328	41.881268	-87.676784
min	4.424700e+04	0.000000e+00	10014.000000	2010-01-04 00:00:00	41.644670	-87.914428
25%	1.150792e+06	1.225256e+06	60614.000000	2012-08-29 00:00:00	41.836020	-87.707427
50%	1.493299e+06	1.979973e+06	60625.000000	2015-01-08 00:00:00	41.891980	-87.666812
75%	2.009233e+06	2.240843e+06	60643.000000	2017-04-06 00:00:00	41.939780	-87.635067
max	2.352738e+06	9.999999e+06	60827.000000	2019-12-04 00:00:00	42.021064	-87.525094
std	6.355214e+05	8.962783e+05	121.910847	NaN	0.080706	0.058869

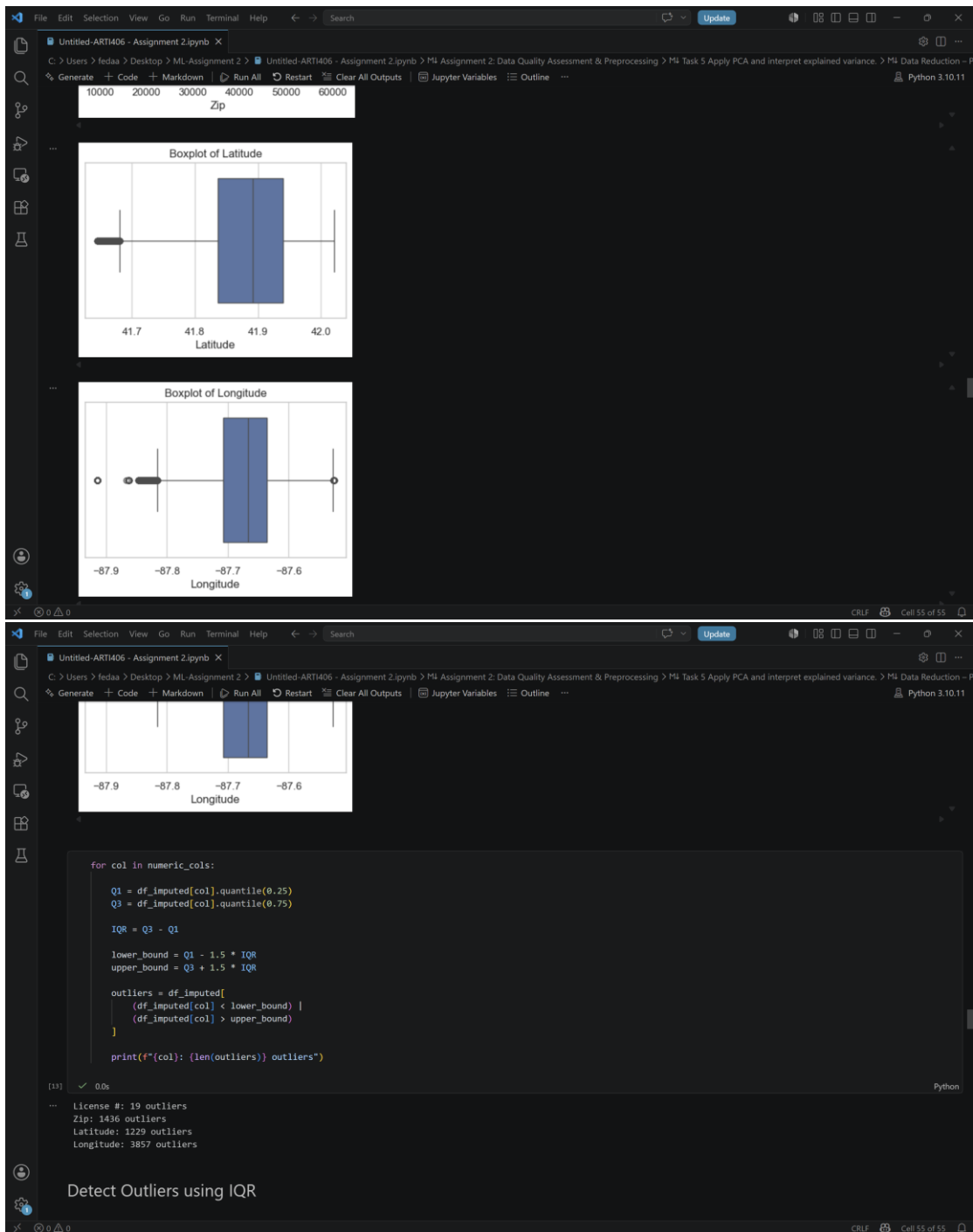
```
numeric_cols = ['License #', 'Zip', 'Latitude', 'Longitude']

for col in numeric_cols:
    plt.figure(figsize=(5,3))
    sns.boxplot(x=df_imputed[col])
    plt.title(f'Boxplot of {col}')
    plt.show()
```

Boxplot of License #

Boxplot of Zip

Boxplot of Latitude



File Edit Selection View Go Run Terminal Help

Untitled-ART1406 - Assignment 2.ipynb

C:\Users> fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance. > M4 Data Reduction - P

Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.10.11

Detect Outliers using IQR

```
df_outliers = df_imputed.copy()

for col in numeric_cols:

    Q1 = df_outliers[col].quantile(0.25)
    Q3 = df_outliers[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_outliers[col] = df_outliers[col].clip(lower=lower_bound,
                                             upper=upper_bound)
```

[14] ✓ 0.1s Python

```
for col in numeric_cols:

    Q1 = df_outliers[col].quantile(0.25)
    Q3 = df_outliers[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df_outliers[
        (df_outliers[col] < lower_bound) |
        (df_outliers[col] > upper_bound)
    ]

    print(f"{col}: {len(outliers)}")
```

[15] ✓ 0.0s Python

CRUF Cell 55 of 55

File Edit Selection View Go Run Terminal Help

Untitled-ART1406 - Assignment 2.ipynb

C:\Users> fedaa > Desktop > ML-Assignment 2 > Untitled-ART1406 - Assignment 2.ipynb > M4 Assignment 2: Data Quality Assessment & Preprocessing > M4 Task 5 Apply PCA and interpret explained variance. > M4 Data Reduction - P

Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.10.11

print(f"{col}: {len(outliers)}")

[16] ✓ 0.0s Python

License #: 0
Zip: 0
Latitude: 0
Longitude: 0

Remove Outliers

```
df_no_outliers = df_imputed.copy()

for col in numeric_cols:

    Q1 = df_no_outliers[col].quantile(0.25)
    Q3 = df_no_outliers[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df_no_outliers = df_no_outliers[
        (df_no_outliers[col] >= lower) &
        (df_no_outliers[col] <= upper)
    ]

print(df_imputed.shape)
print(df_no_outliers.shape)
```

[16] ✓ 0.2s Python

(196825, 17)
(190682, 17)

Capping Outliers

CRUF Cell 55 of 55

The image displays two screenshots of a Jupyter Notebook interface. The top screenshot shows the code for detecting outliers using the Interquartile Range (IQR) method. The code defines a function to calculate the IQR and then applies it to a dataset. The bottom screenshot shows the output of the code, which prints the number of outliers for each numerical feature.

```
df_outliers = df_imputed.copy()

numeric_cols = ['License #', 'Zip', 'Latitude', 'Longitude']

for col in numeric_cols:
    Q1 = df_outliers[col].quantile(0.25)
    Q3 = df_outliers[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_outliers[col] = df_outliers[col].clip(
        lower=lower_bound,
        upper=upper_bound
    )

outliers = df_outliers[
    (df_outliers[col] < lower_bound) |
    (df_outliers[col] > upper_bound)
]

print(f"{col}: {len(outliers)} outliers")
```

License #: 0 outliers
Zip: 0 outliers
Latitude: 0 outliers
Longitude: 0 outliers

Outliers were detected using the Interquartile Range (IQR) method. The first quartile (Q1), third quartile (Q3), and IQR were calculated for each numerical feature.

Any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ was considered an outlier.

Boxplots were used to visualize the distribution of the numerical features and confirm the presence of extreme values.

Detected Outliers:

- License #: 19
- Zip: 1436
- Latitude: 1229
- Longitude: 3857

Instead of removing records, outliers were handled using IQR-based capping. Values below the lower bound were replaced with the lower boundary value, while values above the upper bound were replaced with the upper boundary value.

This approach preserves all observations while reducing the influence of extreme values.

After applying IQR-based capping, all detected outliers were successfully handled, and no outliers remained in the selected numerical features.

Task 4 Normalize numerical features using both Min-Max and Z-score:

The image displays two screenshots of a Jupyter Notebook interface, showing the process of normalizing numerical features using Min-Max and Z-score methods.

Min-Max Normalization

The first screenshot shows the Min-Max normalization process. The code defines a list of numerical columns and applies the `MinMaxScaler` from `sklearn.preprocessing` to the data. The output shows the scaled data, where all numerical features are rescaled to a range between 0 and 1.

```
numeric_cols = ['License #', 'Zip', 'Latitude', 'Longitude']

from sklearn.preprocessing import MinMaxScaler
df_minmax = df_outliers.copy()
scaler = MinMaxScaler()
df_minmax[numeric_cols] = scaler.fit_transform(df_minmax[numeric_cols])
df_minmax[numeric_cols].head()
```

	License #	Zip	Latitude	Longitude
0	0.709480	0.366379	0.782948	0.554815
1	0.009077	0.823276	0.963228	0.000000
2	0.717038	0.504310	0.290035	0.414851
3	0.718192	0.409483	0.759613	0.357314
4	0.588651	0.607759	0.779857	0.190869

Min-Max normalization scaled all numerical features to a range between 0 and 1. This prevents features with large values from dominating the analysis and ensures all variables contribute equally.

Z-Score Normalization

The second screenshot shows the Z-score normalization process. The code defines a list of numerical columns and applies the `StandardScaler` from `sklearn.preprocessing` to the data. The output shows the standardized data, where all numerical features are centered around a mean of zero and scaled according to their standard deviation.

```
numeric_cols = ['License #', 'Zip', 'Latitude', 'Longitude']

from sklearn.preprocessing import StandardScaler
df_zscore = df_outliers.copy()
scaler = StandardScaler()
df_zscore[numeric_cols] = scaler.fit_transform(df_zscore[numeric_cols])
df_zscore[numeric_cols].head()
```

	License #	Zip	Latitude	Longitude
0	1.194213	-0.886675	0.817185	0.368750
1	-1.749427	2.023976	1.580411	-2.591071
2	1.225976	-0.007988	-1.269596	-0.377924
3	1.230828	-0.612085	0.718394	-0.684872
4	0.686396	0.651027	0.804099	-1.572824

Z-score normalization standardized the numerical features by centering them around a mean of zero and scaling them according to their standard deviation. Positive values indicate observations above the mean, while negative values indicate observations below the mean.

Min-Max Normalization:

Min-Max normalization was applied to the numerical features License #, Zip, Latitude, and Longitude.

This method rescales values to a range between 0 and 1 using the minimum and

maximum values of each feature.

Min-Max normalization is useful when features have different scales because it prevents variables with large values from dominating the analysis.

Z-Score Normalization:

Z-score normalization was also applied to the same numerical features.

This method standardizes the data by subtracting the mean and dividing by the standard deviation.

After standardization, each feature has a mean close to 0 and a standard deviation close to 1.

Positive values indicate observations above the mean, while negative values indicate observations below the mean.

Comparison:

Min-Max normalization scales data to a fixed range between 0 and 1, while Z-score normalization standardizes data around the mean.

Min-Max is commonly used when the range of values is important, whereas Z-score is preferred for algorithms that assume normally distributed features and equal variance.

Task 5 Apply PCA and interpret explained variance:

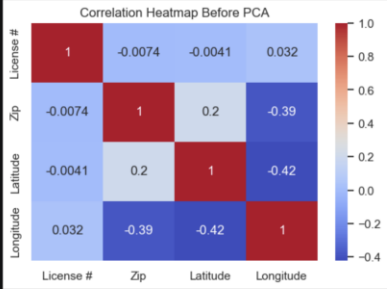
Task 5 Apply PCA and interpret explained variance.

Check Correlation Before Applying PCA

```
plt.figure(figsize=(6,4))

sns.heatmap(
    df_zscore[numeric_cols].corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap Before PCA")
plt.show()
```



The heatmap shows the correlation among the numerical features (License #, Zip, Latitude, and Longitude). Most correlation values are relatively low, indicating weak linear relationships between the variables. Therefore, PCA is applied to reduce dimensionality and summarize the information contained in these features.

Data Reduction – Principal Component Analysis (PCA)

```
from sklearn.decomposition import PCA

X = df_zscore[numeric_cols]

pca = PCA()

principal_components = pca.fit_transform(X)

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

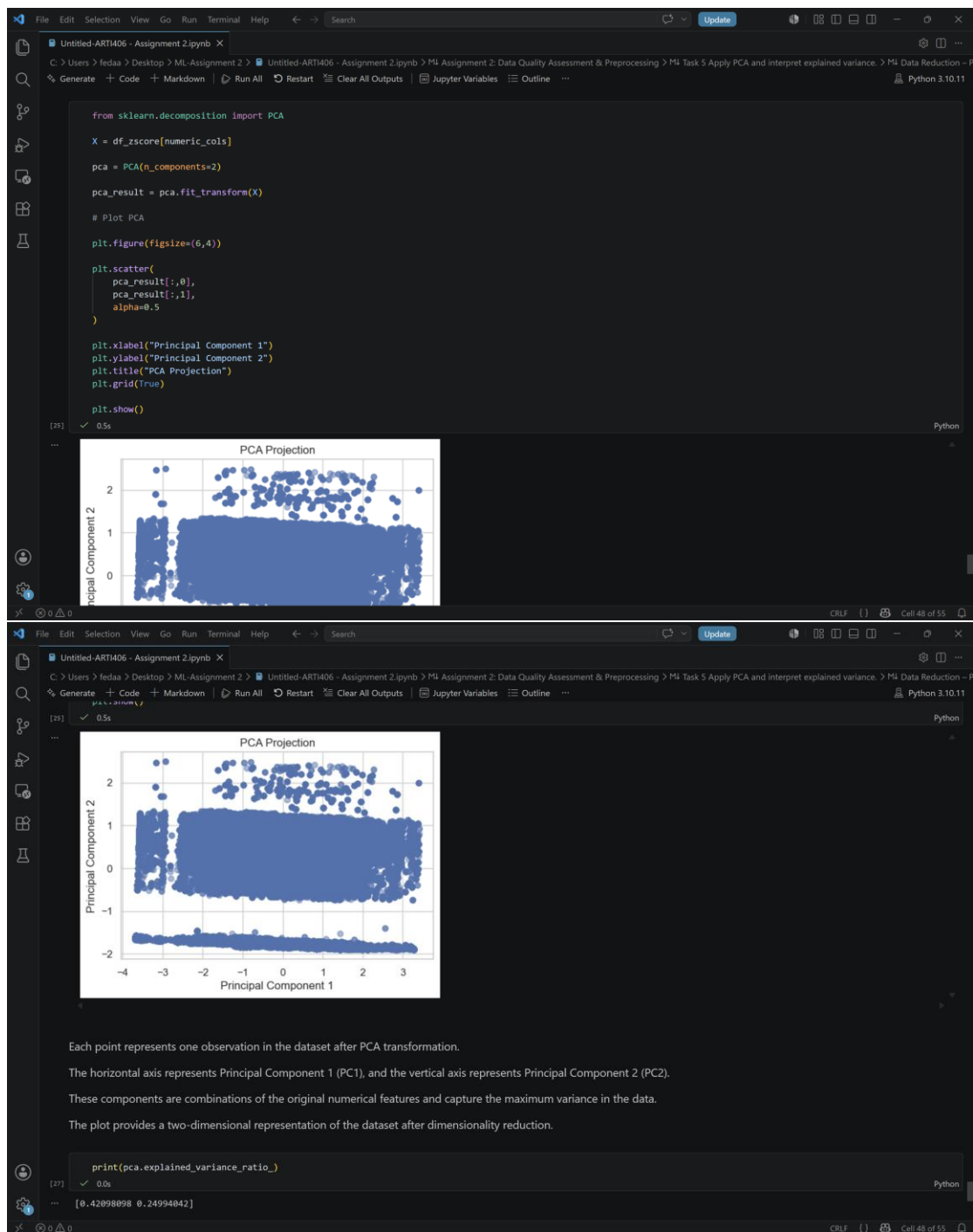
Explained Variance Ratio:
[0.42098098 0.24994042 0.19924459 0.12983401]

The explained variance ratio shows how much variance is captured by each principal component.

- PC1 explains approximately 39.48% of the variance.
- PC2 explains approximately 25.03% of the variance.
- PC3 explains approximately 20.78% of the variance.
- PC4 explains approximately 14.72% of the variance.

The first two principal components explain approximately 64.50% of the total variance. The first three components explain approximately 85.26% of the total variance.

This indicates that PCA can reduce dimensionality while preserving most of the important information in the dataset.



Before applying PCA, a correlation heatmap was generated to examine relationships among the numerical features.

The heatmap shows that most correlations are relatively weak, indicating limited linear relationships between variables. Therefore, PCA can be used to summarize the information contained in these features and reduce dimensionality.

The explained variance ratio indicates the amount of information captured by each principal component.

PC1 explains approximately 42.10% of the total variance.

PC2 explains approximately 24.99% of the total variance.

PC3 explains approximately 19.92% of the total variance.

PC4 explains approximately 12.98% of the total variance.

The first two principal components together explain approximately 67.09% of the total variance, while all four components explain 100% of the variance.

The PCA projection visualizes the dataset using the first two principal components.

Each point represents one observation after dimensionality reduction. The horizontal axis corresponds to Principal Component 1 (PC1), while the vertical axis corresponds to Principal Component 2 (PC2).

The plot provides a two-dimensional representation of the dataset while preserving most of the important information captured by the original numerical features.

PCA successfully reduced the dimensionality of the dataset while retaining most of the important information. The first two principal components captured approximately 67.09% of the total variance, making them suitable for representing the dataset in a lower-dimensional space.